

Intelligent Timetable Generation using Genetic Algorithm

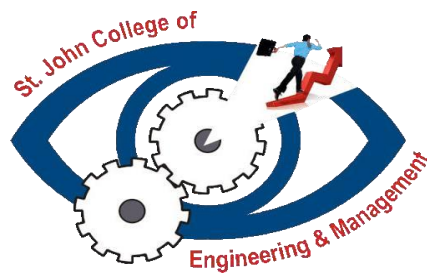
Submitted in partial fulfilment of the
requirements of the degree of
Bachelor of Engineering (B.E.)

In
INFORMATION TECHNOLOGY

By

Raviprakash Gupta	EU2214008
Suryanarayan Pillai	EU1224019
Ayush Raut	EU1224004

Under the guidance of
Mr. Pankaj Kumar
Assistant Professor



Department of Information Technology
St. John College of Engineering and Management, Palghar
AUTONOMOUS INSTITUTE

University of Mumbai

2025–2026

CERTIFICATE

This is to certify that the B.E. project entitled “**Intelligent Timetable Generation using Genetic Algorithm**” is a Bonafide work of **Raviprakash Gupta, Suryanarayan Pillai and Ayush Raut** submitted to University of Mumbai in partial fulfilment of the requirement for the award of the degree of “**Bachelor of Engineering**” in “**Information Technology**” during the academic year 2025–2026.

Mr. Pankaj Kumar

Project Guide

Dr. Arun Saxena

Head of Department

Dr. Kamal Shah

Principal

B.E. Project Report Approval

This project report entitled “**Intelligent Timetable Generation using Genetic Algorithm**” by **Raviprakash Gupta, Suryanarayan Pillai, and Ayush Raut** is approved for the degree of *Bachelor of Engineering* in *Information Technology* from *University of Mumbai*.

Examiners

1. _____

2. _____

Date:

Place:

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Signature

Raviprakash Gupta (EU2214008)

Signature

Suryanarayan Pillai (EU1224019)

Signature

Ayush Raut (EU1224004)

Date:

Place:

ABSTRACT

Timetable generation is a critical and challenging task in educational institutions, involving the allocation of lectures and practical sessions across multiple academic years while satisfying a variety of constraints. Manual timetable creation is not only time-consuming but also prone to errors, conflicts, and inefficiencies, particularly when dealing with multiple batches, shared faculty, and limited classroom and laboratory resources. While lecture sessions can often be accommodated in larger classrooms, practical sessions are inherently more complex due to limited lab capacity, faculty supervision requirements, and the need for batch formation. As a result, practical scheduling becomes the most constrained component of the timetabling process.

The proposed Intelligent Timetable Generation System automates and optimizes this process using a hybrid algorithmic approach. Practical sessions are first allocated using a Backtracking Algorithm, ensuring that hard constraints such as lab capacity, faculty availability, and batch assignments are strictly satisfied. Once practical scheduling is fixed, a Genetic Algorithm (GA) is employed for lecture scheduling to minimize clashes, reduce idle periods, balance faculty workload, and optimize schedule efficiency across multiple academic years.

The system is capable of generating timely, conflict-free, and optimized timetables for n number of academic years while efficiently utilizing shared resources such as classrooms, laboratories, and faculty. Testing confirms that the proposed approach produces accurate, efficient, and reliable timetables, significantly reducing manual effort and scheduling errors.

Keywords: *Timetable scheduling, curriculum-based scheduling, Genetic Algorithm (GA), Backtracking algorithm, Laboratory scheduling, Automation, Educational Systems*

Table of Contents

	Abstract	v
	List of Figures	ix
	List of Tables	x
	List of Abbreviations	xi
Chapter 1	Introduction	1
	1.1 Motivation	2
	1.2 Problem Statement	2
	1.3 Objectives	3
	1.4 Scope	3
	1.5 Summary	4
Chapter 2	Review of Literature	5
	2.1 Literature Survey	5
	2.2 Research Gap	5
	2.3 Proposed System Contribution	6
	2.6 Summary	6
Chapter 3	Requirement Analysis	7
	3.1 Hardware Requirements	7
	3.2 Software Requirements	7
	3.3 Functional Requirements	8
	3.4 Non-Functional Requirements	8
	3.5 Summary	8
Chapter 4	System Design	9
	4.1 DFD Diagrams	9

4.1.1 Level 0 DFD	9
4.1.2 Level 1 DFD	10
4.1.3 Level 2 DFD	11
4.2 UML Diagrams	14
4.3 Summary	19
Chapter 5	Report on Present Investigation
5.1 Proposed System	20
5.1.1 Block Diagram of Proposed System	21
5.1.2 System Flowcharts	22
5.1.2.1 Practical Scheduling Flowchart	22
5.1.2.2 Lecture Scheduling Flowchart	24
5.1.2.3 Combined Flowchart	25
5.2 Methodology	26
5.2.1 Two-Phase Scheduling Strategy	26
5.2.2 Justification of Approach	26
5.2.3 Constraint Modelling	26
5.3 Summary	27
Chapter 6	Results and Discussion
6.1 System Testing and Evaluation	28
6.1.1 Functional Testing	28
6.1.2 Performance Analysis	28
6.2 Analysis of Results	29
6.3 Screenshots of Working Model	29
6.4 Summary	34

Chapter 7	Conclusion and Future Scope	35
	7.1 Summary of Findings	35
	7.2 Future Scope and Possible Enhancements	35
	References	36
	Appendix	37
	Acknowledgement	39

List of Figures

Figure No.	Figure Name	Page No.
Fig. 4.1.1	Level 0 Data Flow Diagram of Timetable Generator	9
Fig. 4.1.2	Level 1 Data Flow Diagram of Timetable Generator	11
Fig. 4.1.3	Level 2 Data Flow Diagram of Timetable Generator	13
Fig. 4.2.1	Use Case Diagram of Timetable Generator	14
Fig. 4.2.2	Class Diagram of Timetable Generator	15
Fig. 4.2.3	Activity Diagram of Timetable Generator	16
Fig. 4.2.4	Sequence Diagram of Timetable Generator	17
Fig. 4.2.5	Gantt Chart	19
Fig. 5.1.1	Block Diagram of Proposed System Architecture	21
Fig. 5.1.2	Flowchart of Practical Scheduling using Backtracking	23
Fig. 5.1.3	Flowchart of Lecture Scheduling using GA	24
Fig. 5.1.4	Flowchart of entire process	25
Fig. 6.3.1	Homepage	29
Fig. 6.3.2	Department details Input form	30
Fig. 6.3.3	Department Dashboard	31
Fig. 6.3.4	Manage Years Page	32
Fig. 6.3.5	Manage Timetable Page	33
Fig. 6.3.6	Generated Timetable Page	33

List of Tables

Table No.	Table Name	Page No.
Table 3.1	Hardware Requirements	7
Table 3.2	Software Requirements	7
Table 4.2	Project Development Timeline	18
Table 6.1	Execution time Table	28

List of Abbreviations

Sr.No.	Abbreviation	Meaning
1	AI	Artificial Intelligence
2	API	Application Programming Interface
3	CSS	Cascading Style Sheets
4	DEAP	Distributed Evolutionary Algorithms in Python
5	DFD	Data Flow Diagram
6	FR	Functional Requirement
7	GA	Genetic Algorithm
8	GUI	Graphical User Interface
9	HTML	HyperText Markup Language
10	HTTP	Hypertext Transfer Protocol
11	JSON	JavaScript Object Notation
12	ORM	Object Relational Mapping
13	PDF	Portable Document Format
14	SQLite	Structured Query Language Lite
15	UML	Unified Modelling Language

CHAPTER 1

INTRODUCTION

Timetable generation is one of the most essential yet challenging administrative tasks in educational institutions. It involves assigning lectures, practical sessions, instructors, classrooms, and laboratories to available time slots while satisfying multiple institutional constraints. In curriculum-based academic programs, timetables must often be prepared simultaneously for multiple academic years, which significantly increases the complexity of the problem. As the number of subjects, faculty members, classrooms, laboratories, and student divisions grows, manual timetable preparation becomes increasingly difficult, time-consuming, and prone to errors such as overlapping classes, faculty conflicts, and inefficient resource utilization.

Among all scheduling components, practical session allocation is the most constrained and difficult part of the timetabling process. While lecture sessions can usually be accommodated in larger classrooms, practical sessions require specialized laboratories with limited capacity and infrastructure. To manage this, students of a single academic year are divided into smaller batches for laboratory sessions to ensure effective supervision and better student–teacher interaction. Since lecture sessions require the entire class to attend together, these batches cannot be scheduled independently, making practical scheduling significantly more complex than lecture scheduling.

To address these challenges, this project, titled **“Intelligent Timetable Generation using Genetic Algorithm”**, proposes a hybrid approach for automatically generating optimal timetables. First, a Backtracking Algorithm is used to allocate practical and laboratory sessions while strictly satisfying hard constraints such as laboratory capacity, faculty availability, batch allocation, and required teaching hours. After practical scheduling is finalized, an occupancy matrix is created to track the usage of resources and time slots.

Once the practical sessions are fixed, a Genetic Algorithm (GA) is applied for lecture scheduling. Using the principles of selection, crossover, and mutation, the GA evolves multiple timetable configurations to generate an efficient and conflict-free schedule. The integration of backtracking and GA provides both strict constraint satisfaction and timetable optimization, resulting in a scalable, flexible, and intelligent scheduling system that reduces administrative workload and ensures effective utilization of institutional resources.

1.1 Motivation

The motivation behind developing an intelligent timetable generation system arises from the numerous challenges associated with traditional manual scheduling methods in educational institutions. Timetable preparation is a complex and time-consuming task, particularly in large institutions that offer multiple courses across different academic years and have a large number of faculty members, classrooms, and laboratories. Preparing such schedules manually requires significant administrative effort and often becomes tedious and inefficient.

Manual timetable creation is also highly prone to human errors. Common issues such as overlapping classes, double-booking of classrooms or laboratories, incorrect faculty assignments, and inefficient time slot allocation frequently occur during manual scheduling. In addition, managing multiple constraints such as teacher availability, classroom capacity, laboratory limitations, batch divisions, and subject-specific requirements makes the process even more difficult. As the number of constraints increases, maintaining accuracy and consistency in the timetable becomes a major challenge.

Another important concern is the uneven distribution of workload among faculty members, which can affect both productivity and teaching efficiency. Furthermore, any changes in faculty availability, subject allocation, or institutional requirements often demand repeated manual adjustments, increasing both complexity and time consumption. These limitations highlight the growing need for an automated and intelligent solution capable of handling complex scheduling problems efficiently.

This project is therefore motivated by the need to develop a smart, reliable, and optimized timetable generation system using Artificial Intelligence techniques. By automating the scheduling process through intelligent algorithms, the system aims to reduce administrative workload, improve accuracy, minimize conflicts, and ensure efficient utilization of institutional resources.

1.2 Problem Statement

The academic timetable generation process remains fundamentally complex and highly constrained. Educational institutions must allocate lectures, practical sessions, faculty members, classrooms, and laboratories across multiple academic years while satisfying numerous academic and resource-based constraints. Traditional manual scheduling methods are time-consuming, error-prone, and often result in clashes such as overlapping classes, double-booking of

laboratories, and faculty conflicts. The complexity further increases in curriculum-based programs where practical sessions require batch-wise division of students due to limited laboratory capacity and supervision requirements. These practical sessions must be assigned fixed-duration time slots without overlap, while ensuring proper allocation of laboratories, faculty, and academic hours. In addition, maintaining predefined lecture–practical relationships and avoiding conflicts in shared resources significantly restrict scheduling flexibility. Existing timetable generation approaches often fail to effectively handle batch-based practical scheduling, multi-year coordination, and real-world institutional constraints in a scalable and efficient manner. There is a clear need for an intelligent automated scheduling system capable of generating conflict-free, optimized, and resource-efficient timetables while satisfying all hard academic constraints.

1.3 Objectives

The primary objectives of this project are:

1. To design and implement a web-based application using Django, Python, SQLite, Javascript, React CSS, HTML for easy timetable generation, management, and access.
2. To generate conflict-free schedules for lectures and practical sessions by avoiding clashes in faculty allocation, classroom usage, and laboratory occupancy.
3. To efficiently schedule batch-wise practical sessions by considering laboratory capacity.
4. To allocate practical sessions using a Backtracking Algorithm for strict constraint satisfaction.
5. To optimize lecture scheduling using a Genetic Algorithm (GA) to improve timetable efficiency and reduce scheduling conflicts.
6. To minimize manual effort, reduce scheduling errors, and improve administrative efficiency through automation.
7. To provide a user-friendly interface for administrators to input data, generate timetables, and view schedules.

1.4 Scope

The scope of this project includes the design and development of an intelligent timetable generation system for a single college department covering multiple academic years, specifically 2nd year, 3rd year, and 4th year students. The system is capable of scheduling both lecture sessions and practical/laboratory sessions while satisfying academic and resource-related constraints.

A major focus of the project is the efficient handling of batch-wise practical scheduling, where each class is divided into three batches for laboratory sessions. The system considers essential constraints such as faculty availability, laboratory capacity, classroom allocation, fixed subject requirements, and time slot availability to ensure conflict-free scheduling. It aims to prevent clashes in faculty assignments, classroom usage, and laboratory occupancy while maintaining a balanced and optimized timetable.

The project also includes the development of a web-based application using Django, Python, and SQLite, which provides a user-friendly interface for administrators to generate, view, and modify timetables efficiently. Furthermore, the system is designed with future scalability in mind and can be extended to support additional departments, dynamic timetable updates, and more advanced scheduling constraints.

1.5 Summary

This chapter presented the motivation, problem statement, objectives, and scope of the Intelligent Timetable Generation System. It emphasized the limitations of manual timetable preparation, including time consumption, scheduling conflicts, and inefficient resource utilization.

To overcome these challenges, the system adopts a hybrid approach using Backtracking Algorithm for practical scheduling and Genetic Algorithm (GA) for lecture allocation. This method ensures conflict-free timetables while significantly reducing timetable generation time.

The system is implemented as a web-based application using Django, Python, and SQLite, providing an efficient platform for timetable generation and management within a single department.

Key Highlights:

- Faster timetable generation through automation
- Conflict-free lecture and practical scheduling
- Hybrid use of Backtracking + Genetic Algorithm
- Efficient allocation of faculty, classrooms, and laboratories
- Web-based implementation using Django and Python

CHAPTER 2

REVIEW AND LITERATURE SURVEY

2.1 Literature Survey

Timetable generation is a complex task in academic institutions due to multiple constraints such as faculty availability, classrooms, laboratories, and time slots. Manual scheduling is time-consuming and often leads to conflicts, which has led to the development of automated systems.

Genetic Algorithms (GA) are widely used for timetable generation. Mittal et al. [1] showed that GA can effectively reduce conflicts and improve scheduling by exploring multiple solutions. Puttaswamy et al. [2] combined GA with heuristic techniques to improve resource allocation, although their approach depends on predefined structures.

Romaguera et al. [3] proposed an enhanced GA-based system to reduce infeasible schedules and improve efficiency. Sarnaik et al. [4] focused on handling real-world constraints, making the system more practical. Katore et al. [5] introduced a hybrid approach, showing improved performance but increased complexity.

Overall, existing methods are effective but struggle with complex constraints, especially in practical scheduling and multi-year systems. This highlights the need for simpler and more structured approaches.

2.2 Research Gap

Despite the progress in automated timetable generation, several limitations still exist in current approaches:

- Many systems rely mainly on Genetic Algorithms, which are not very effective for handling highly constrained tasks such as practical and laboratory scheduling.
- Most approaches treat all scheduling tasks in the same way, without separating practical sessions (which have strict constraints) from lecture sessions (which are more flexible).
- Limited attention on dividing students class into batches for practicals.
- Some methods depend on fixing conflicts after timetable generation instead of avoiding them during the scheduling process

- Limited attention is given to handling multiple academic years together while sharing resources like faculty and labs.

These limitations show the need for a more structured approach that can handle different types of scheduling more effectively while reducing conflicts.

2.3 Proposed System Contribution

To address these limitations, this project proposes an Intelligent Timetable Generation System using a hybrid approach.

The system follows a two-phase scheduling strategy:

- Phase 1: Practical sessions are scheduled using a Backtracking Algorithm to satisfy strict constraints such as lab availability, batch allocation, and faculty assignment.
- Phase 2: Lecture sessions are scheduled using a Genetic Algorithm to fill the remaining time slots efficiently.

By separating practical and lecture scheduling, the system reduces complexity and improves the overall scheduling process. The system also uses an occupancy matrix to track resource usage and help avoid conflicts during scheduling. Additionally, the system is implemented as a web-based application, allowing users to input data and view generated timetables easily.

2.10 Summary

The literature shows that Genetic Algorithms and hybrid methods are effective for timetable generation, but existing approaches often struggle with highly constrained components like practical scheduling. To address this, the proposed system uses a hybrid approach combining backtracking for practicals and a Genetic Algorithm for lectures, ensuring efficient and conflict-free timetable generation.

CHAPTER 3

REQUIREMENTS ANALYSIS

3.1 Hardware Requirements

The system requires minimal hardware as it is lightweight and can run on standard machines.

Table 3.1: Hardware Requirements

Sr.No.	Component	Minimum Specification
1	Processor	Intel Core i5 (8th Gen) or AMD Ryzen 5 equivalent
2	RAM	8 GB
3	Storage	2 GB free space

3.2 Software Requirements

Table 3.2: Software Requirements

Sr.No.	Software / Service	Version / Specification
1	Programming Language	Python
2	Frontend Framework	React, Charts.js
3	Backend Framework	Django
3	Technologies	Javascript, HTML, CSS
4	Database	SQLite
5	Library	DEAP, Charts.js, Numpy
6	Operating System	Windows / Linux / macOS

3.3 Functional Requirements

FR-01: The system shall allow users to input data such as teachers, classrooms, laboratories, subjects, and academic years.

FR-02: The system shall generate practical timetables using a backtracking algorithm.

FR-03: The system shall generate lecture schedules using a Genetic Algorithm.

FR-04: The system shall ensure no conflicts in teacher, room, or student schedules.

FR-05: The system shall handle multiple academic years simultaneously.

FR-06: The system shall display the generated timetable in a user-friendly format.

3.4 Non-Functional Requirements

NFR-01 - Performance: The system should generate timetables within a few seconds for medium-sized datasets.

NFR-02 - Scalability: The system should handle multiple academic years and increasing data efficiently.

NFR-03 - Usability: The interface should be simple and easy to use.

NFR-04 - Reliability: The system should generate valid timetables without conflicts.

NFR-05 - Maintainability: The code should be modular and easy to update.

3.5 Summary

This chapter outlined the key requirements of the timetable generation system, including hardware and software specifications, core functional features, and essential quality attributes. These requirements serve as the foundation for the system design and implementation discussed in the following chapters.

CHAPTER 4

System Design

4.1 Data Flow Diagrams

Data Flow Diagrams (DFDs) model the movement of data through the system at increasing levels of detail. Three levels of DFD are presented: the Level 0 Context Diagram, which represents the entire system as a single process; the Level 1 DFD, which decomposes the system in 8 processes; and the Level 2 DFD, which further expands the Lecture Scheduling sub-process into its constituent operations. Together, these three diagrams provide a comprehensive picture of how data originates, is transformed, and is stored throughout the platform.

4.1.1 Level 0 DFD

The Level 0 DFD in Fig. 4.1.1 represents the Timetable Generator System as a single central process interacting with two main external components: the User/Admin and the Database.

The User/Admin provides input data to the system, including department details, academic years, subjects, teachers, classrooms, laboratories, and other scheduling parameters. The system processes this input to generate a timetable. The final generated timetable is then sent back to the User/Admin for viewing and saving.

The system also interacts with the Database in a bidirectional manner. It stores input data such as resources and configurations, and retrieves this data whenever required for timetable generation. Additionally, generated timetables can be saved and later fetched from the database.

Overall, the diagram shows how the system takes input from the user, processes it, stores/retrieves necessary data, and produces the final timetable output.

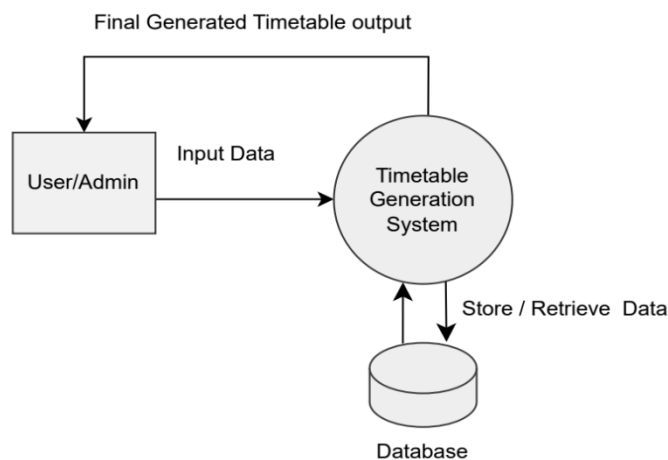


Fig. 4.1.1: Level 0 Data Flow Diagram of Timetable Generator

4.1.2 Level 1 DFD

The Level 1 DFD presented in Fig 4.1.2 decomposes the Timetable Generator System into multiple sub-processes that handle different stages of timetable creation. It includes five data stores: D1 Input Storage, D2 Practical Timetable, D3 Lecture Timetable, D4 Final Timetable, and D5 Saved Timetables.

P1 - Data Input & Validation receives department and academic year data from the User/Admin, validates it, and stores the resource data in D1 Input Storage.

P2 - Feasibility Check reads stored resource data from D1 and verifies whether sufficient resources (teachers, labs, classrooms, time slots) are available to generate a valid timetable. It outputs feasible data for further processing.

P3 - Data Preprocessing / Encoding processes the feasible data and organizes it year-wise into lecture data and practical data formats required for scheduling.

P4 - Practical Scheduling (Backtracking) schedules practical sessions first using a backtracking approach and stores the generated practical timetable in D2.

P5 - Occupancy Matrix Generation creates a matrix using practical data to track occupied resources (teachers, labs, time slots), which helps avoid conflicts during lecture scheduling.

P6 - Lecture Scheduling (Genetic Algorithm) uses the processed lecture data along with the occupancy matrix to schedule lectures efficiently and stores the lecture timetable in D3.

P7 - Timetable Integration combines both lecture and practical timetables from D2 and D3 to generate a complete timetable, which is stored in D4.

P8 - Timetable Management handles saving, retrieving, and managing timetables. It stores final timetables in D5 and provides the final output to the User/Admin.

Overall, the diagram shows a step-by-step flow where validated input data is processed, practicals are scheduled first, lectures are scheduled next, and both are combined to produce and manage the final timetable.

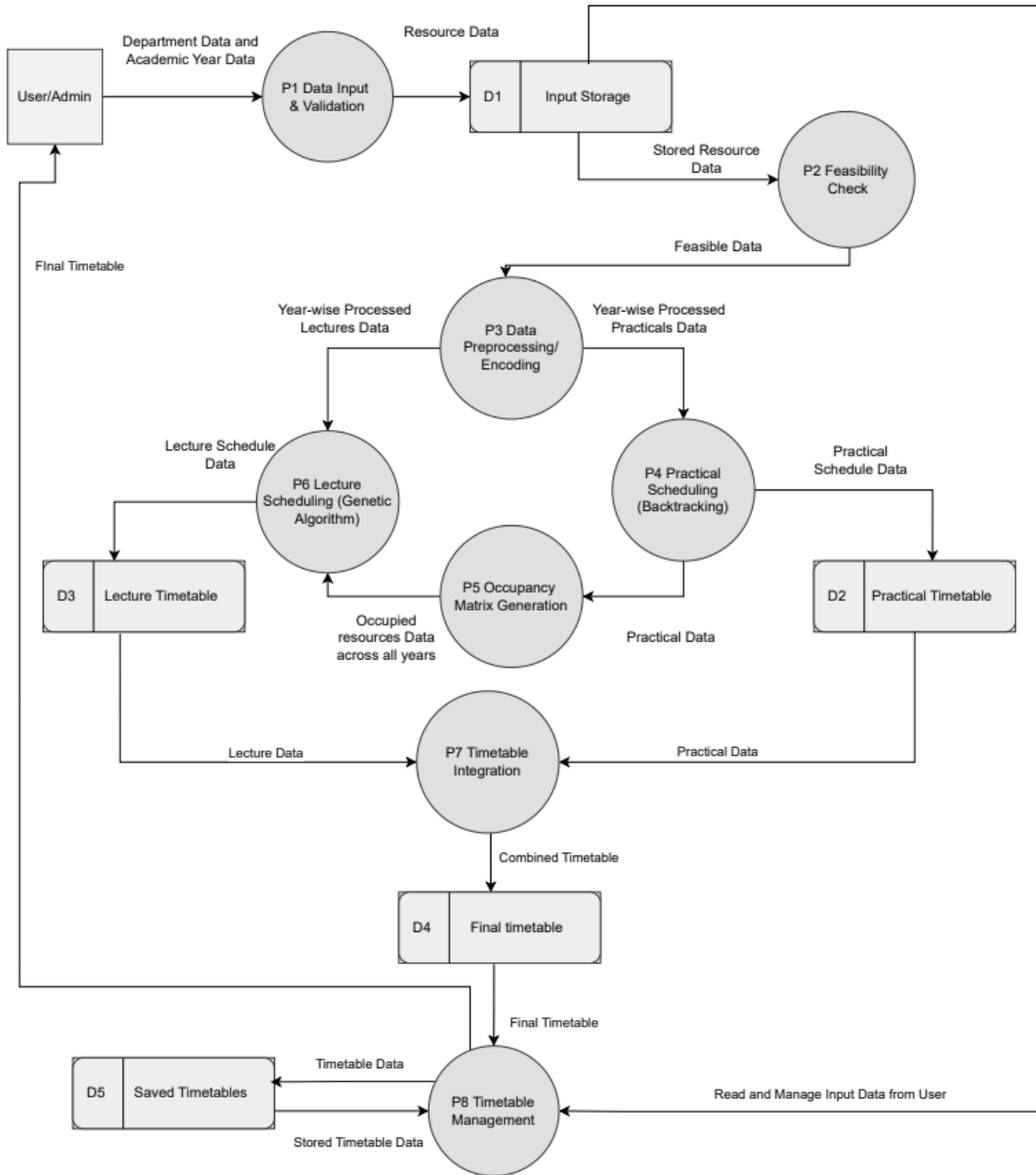


Fig. 4.1.2: Level 1 Data Flow Diagram of Timetable Generator

4.1.3 Level 2 DFD

The Level 2 DFD presented in Fig 4.1.3 expands the Lecture Scheduling process (Genetic Algorithm) into detailed sub-processes, showing how lecture timetables are generated step by step. It uses five data stores: D1 Lectures and Assigned Teachers, D2 Classrooms and Capacity, D3 Additional Lecture Data, D4 Occupancy Matrix, and D5 Final Timetable.

P1 - Fetch Lecture Data reads lecture requirements, assigned teachers, and classroom details from D1, D2, and D3, and prepares the input data needed for scheduling.

P2 - Initial Population Generation creates an initial set of possible timetable solutions (population) based on lecture requirements.

P3 - Selection selects the best candidate timetables (parents) from the population based on their quality for further processing.

P4 - Crossover combines selected parent solutions to generate new offspring timetables by exchanging parts of their schedules.

P5 - Mutation introduces small random changes in the offspring timetables to increase diversity and explore better solutions.

P6 - Fitness Evaluation evaluates each candidate timetable using constraints such as resource conflicts and uses the occupancy matrix from D4 to ensure no clashes occur. It assigns fitness scores to each solution.

P7 - Elitism & Replacement keeps the best solutions and replaces weaker ones with improved candidates to form the next generation.

P8 - Termination Check verifies whether the stopping condition (such as maximum generations or acceptable fitness level) is reached. If not, the process repeats from selection.

Finally, the best timetable generated is stored in D5 as the Final Timetable.

Overall, this diagram shows how the Genetic Algorithm iteratively improves timetable solutions until an optimal or near-optimal lecture schedule is obtained.

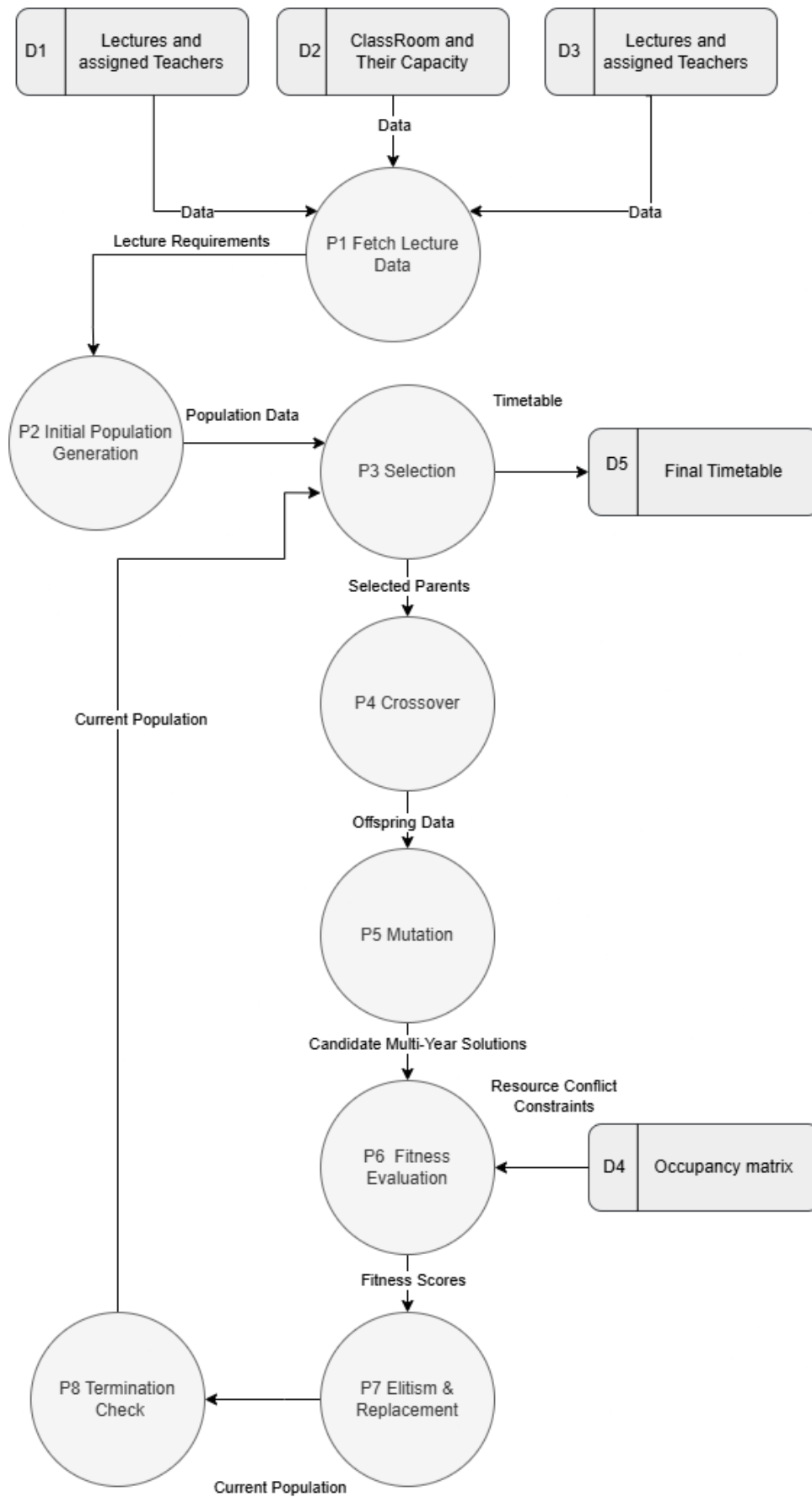


Fig. 4.1.3: Level 2 Data Flow Diagram of Timetable Generator

4.2 UML Diagrams

4.2.1 Use Case Diagram

The Use Case Diagram in Fig 4.2.1 represents the interactions between the User/Admin and the Timetable Generator system. The User/Admin is the main actor who controls all operations in the system.

The system provides several use cases: creating departments, adding department resources (like teachers, classrooms, labs), creating academic years, and adding academic resources (subjects, batches, etc.). The user can also manage all resources in one place.

After setting up the data, the User/Admin can generate the timetable automatically, view the generated timetable, and save it for future use.

Overall, the diagram shows how the User/Admin handles the complete process of setting up data and generating a timetable within the system.

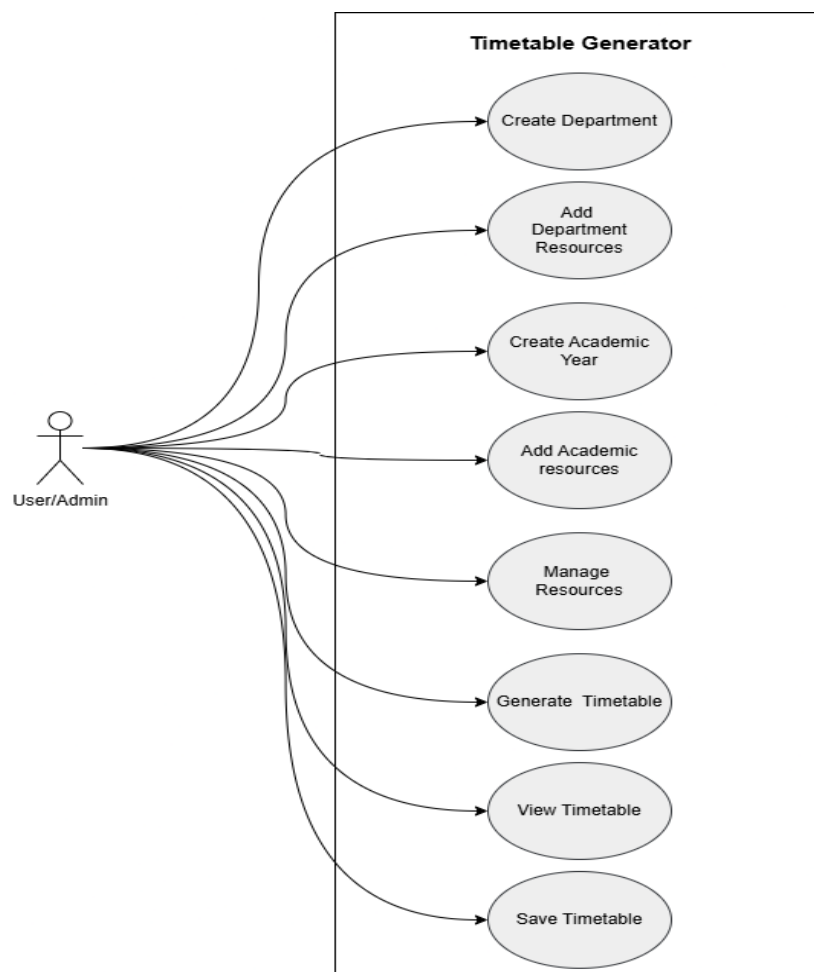


Fig. 4.2.1: Use Case Diagram of Timetable Generator

4.2.2 Class Diagram

The class diagram represents the core components of the timetable generation system and their relationships. The **UserInterface** handles user interactions, while the **TimetableSystem** acts as the central controller, managing data loading and timetable generation. It uses the **Backtracking Algorithm** for practical scheduling and the **Genetic Algorithm** for lecture scheduling. The **Database** manages storage and retrieval of system data. The **Year** and **TimeSlot** classes define the academic structure and scheduling framework. The **Lecture** and **Practical** classes store subject details, while **Teacher**, **ClassRooms**, and **Lab** represent the resources used during scheduling. Key relationships include associations between Year and TimeSlot, TimeSlot and sessions, and the allocation of teachers, classrooms, and labs to lectures and practicals..

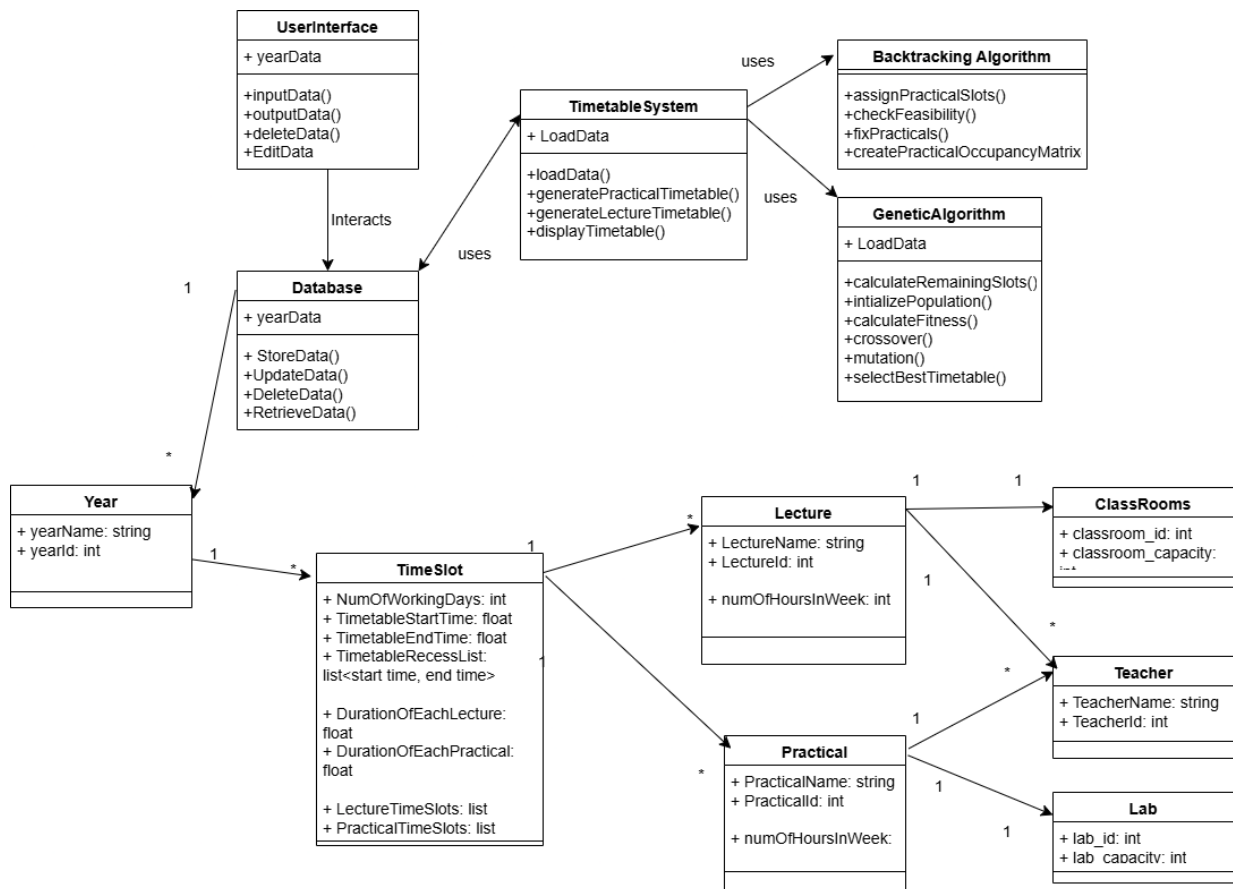


Fig. 4.2.2: Class Diagram of Timetable Generator

4.2.3 Activity Diagram

The activity diagram in Fig 4.2.3 illustrates the overall workflow of the proposed two-phase timetabling system. The process begins with configuring department data, including academic years, subjects, faculty assignments, and required teaching hours. Once the input is defined, a feasibility check is performed to ensure that the available resources are sufficient to generate a valid timetable. If the configuration is not feasible, the system identifies possible reasons and prompts the user to modify the input. When feasible, the scheduling process proceeds in two distinct phases.

In **Phase 1**, practical sessions are allocated using a **backtracking algorithm**, ensuring that all hard constraints such as laboratory capacity, batch synchronization, and faculty availability are satisfied. This phase iteratively assigns practical slots until all required hours are completed, after which the practical timetable is fixed and resource occupancy matrices are generated.

In **Phase 2**, the remaining lecture sessions are scheduled using a **Genetic Algorithm**. The best solution obtained based on the fitness function is selected, and the final timetable is generated by combining both practical and lecture schedules. This structured approach improves efficiency by handling highly constrained components first and reducing the search space for subsequent optimization.

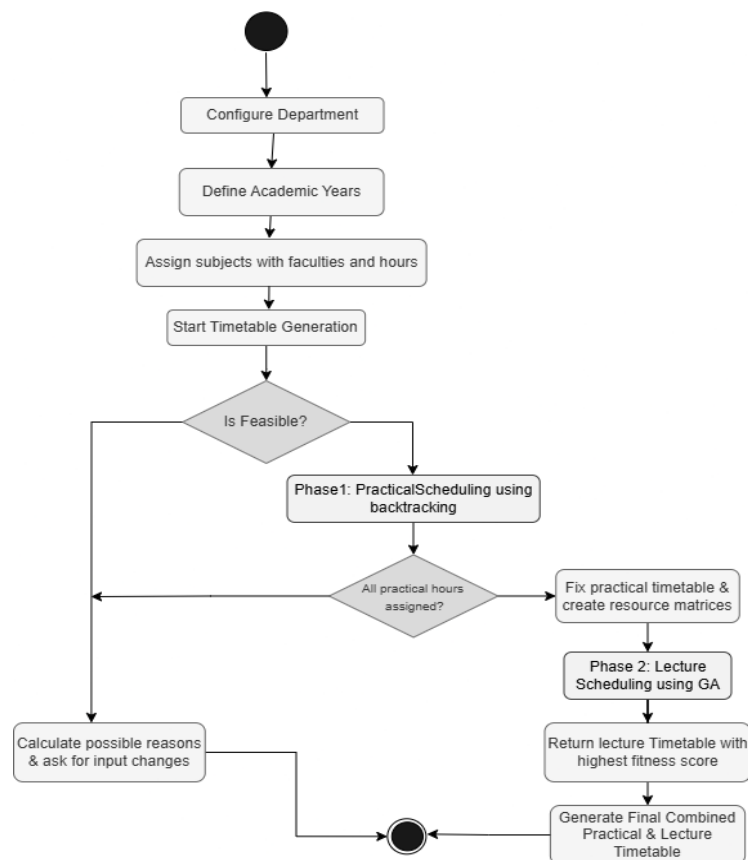


Fig. 4.2.3: Activity Diagram of Timetable Generator

4.2.4 Sequence Diagram

The sequence diagram illustrates the interaction between the User, Frontend, Backend, and Scheduling Engine during timetable generation. Initially, the user enters department-level data, which is sent to the backend for validation and storage. Subsequently, year-wise data is entered iteratively, represented using a loop to support multiple academic years.

Once all required data is configured, the user requests timetable generation. The backend processes this request and forwards the data to the scheduling engine. The scheduling engine first performs practical scheduling using a backtracking algorithm, followed by occupancy matrix generation. It then applies a Genetic Algorithm to optimize lecture scheduling and combines both results to produce the final timetable.

The generated timetable is returned to the backend, stored in the database, and sent to the frontend for display to the user. This sequence ensures efficient handling of multi-year scheduling and demonstrates the hybrid approach used in the system.

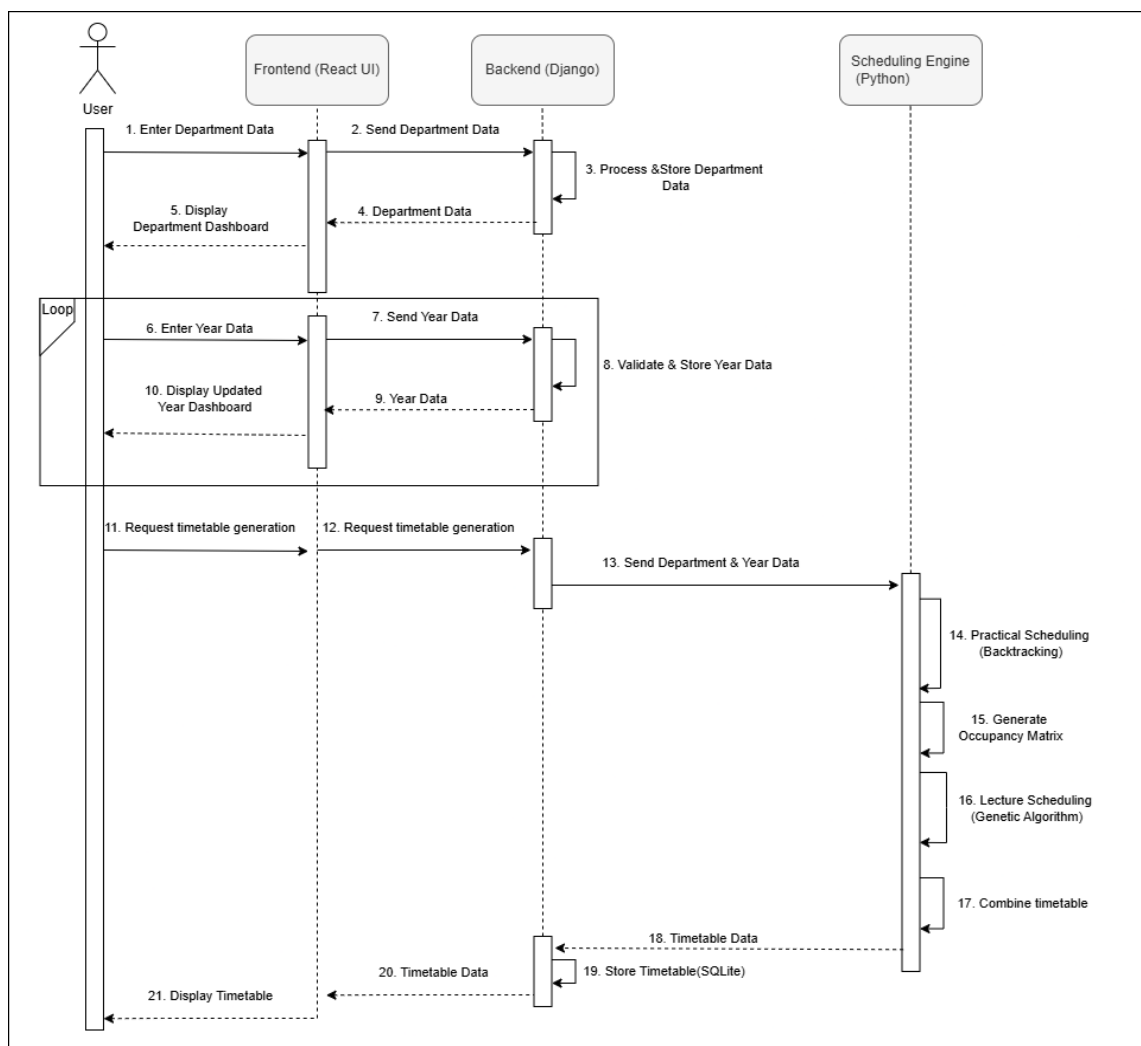


Fig. 4.2.4: Sequence Diagram of Timetable Generator

4.2.5 Timeline / Gantt Chart

The project development timeline is illustrated in Fig. 4.2.5 as a Gantt chart spanning 16 weeks.

Table 4.2: Project Development Timeline

Sr. No	Task / Module	Timeline	Deliverable
1	Requirements gathering and competitive analysis	Week 1–2	Requirements specifications
2	System design and architecture planning	Week 3	Architecture diagrams and workflow design
3	Project setup (Django backend + React frontend)	Week 4	Initial Project Structure
4	Input modules (Department and Year data forms)	Week 5-6	Functional Data input interface
5	Feasibility checking module	Week 7	Resource validation
6	Practical scheduling using backtracking	Week 8 -10	Conflict free practical timetable
7	Lecture scheduling using Genetic Algorithm (DEAP)	Week 11-13	Optimized lecture timetable
8	Integration of scheduling modules	Week 14	Complete timetable
9	Testing and performance evaluation	Week 15	Test results and execution analysis
10	Documentation	Week 16	Project Report & research paper

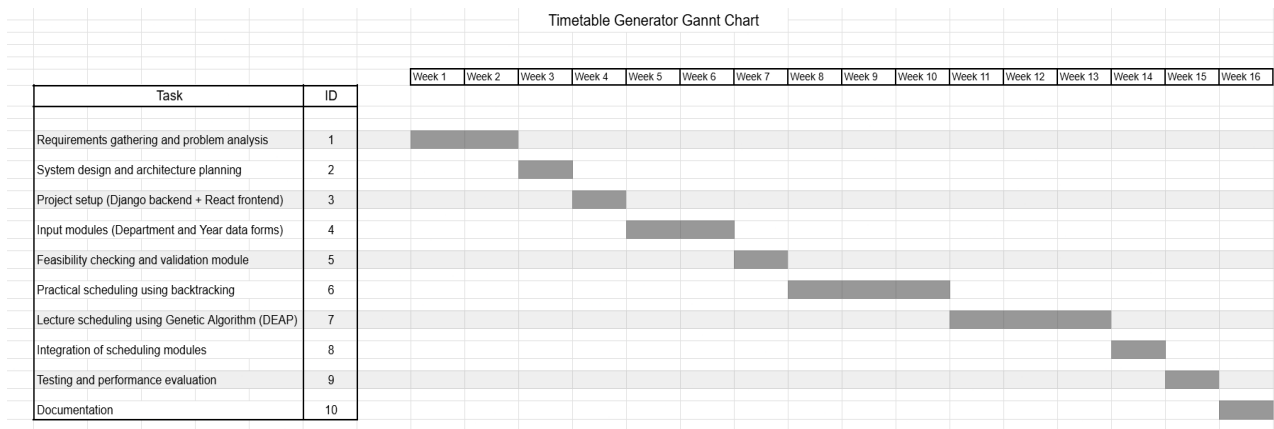


Fig. 4.2.5: Gantt Chart of Project Development Timeline

4.3 Summary

This chapter presented the design of the Timetable Generator system. It explained the system architecture, flowcharts for practical and lecture scheduling, and the overall working process. The constraint modelling defined the rules required to generate a valid timetable.

The project timeline and Gantt chart described how the development was planned and executed. Together, these design elements provide a clear link between the system requirements and its implementation.

CHAPTER 5

REPORT ON PRESENT INVESTIGATION

5.1 Proposed System

The Timetable Generator follows a layered architecture as illustrated in Fig. 5.1, consisting of five main layers: Client Layer, React Frontend, Django Backend, Scheduling Engine, and Data Layer.

- Client Layer - This layer represents user interactions such as creating departments, defining academic years, managing resources, generating timetables, and viewing saved timetables.
- Frontend Layer (React) - Implemented using React, this layer provides modules such as Dashboard, Department Management, Year Management, Timetable Viewing, and Visualization. It handles user inputs and displays outputs in an interactive manner.
- Backend Layer (Django) - The Django backend processes requests from the frontend through components such as the Request Handler, Input Validator, and Controller. It ensures proper validation and coordinates with the scheduling engine.
- Scheduling Engine (Python) - This is the core of the system where timetable generation takes place. It includes:
 - Feasibility Checker to verify resource availability
 - Phase 1: Practical Scheduler using Backtracking
 - Resource Tracker using occupancy matrix
 - Phase 2: Lecture Scheduler using Genetic Algorithm (DEAP)
- Data Layer (SQLite) - Stores all system data including department resources, academic year resources, and saved timetables. It supports efficient data retrieval and storage through Django ORM.

5.1.1 Block Diagram of Proposed System

The block diagram in Fig 5.1.1 illustrates the layered architecture of the proposed academic timetabling system. The Client Layer includes user actions such as creating departments, managing resources, defining academic years, and generating timetables. These interact with the React-based Frontend Layer, which provides the user interface and visualizes results using Chart.js. The Django-based Backend Layer handles request processing and coordinates the scheduling operations. The Core Scheduling Engine, implemented in Python, performs feasibility checking, practical scheduling using backtracking, and lecture scheduling using a Genetic Algorithm implemented with DEAP. The Data Layer uses SQLite to store all system data and generated timetables, enabling efficient and modular system execution.

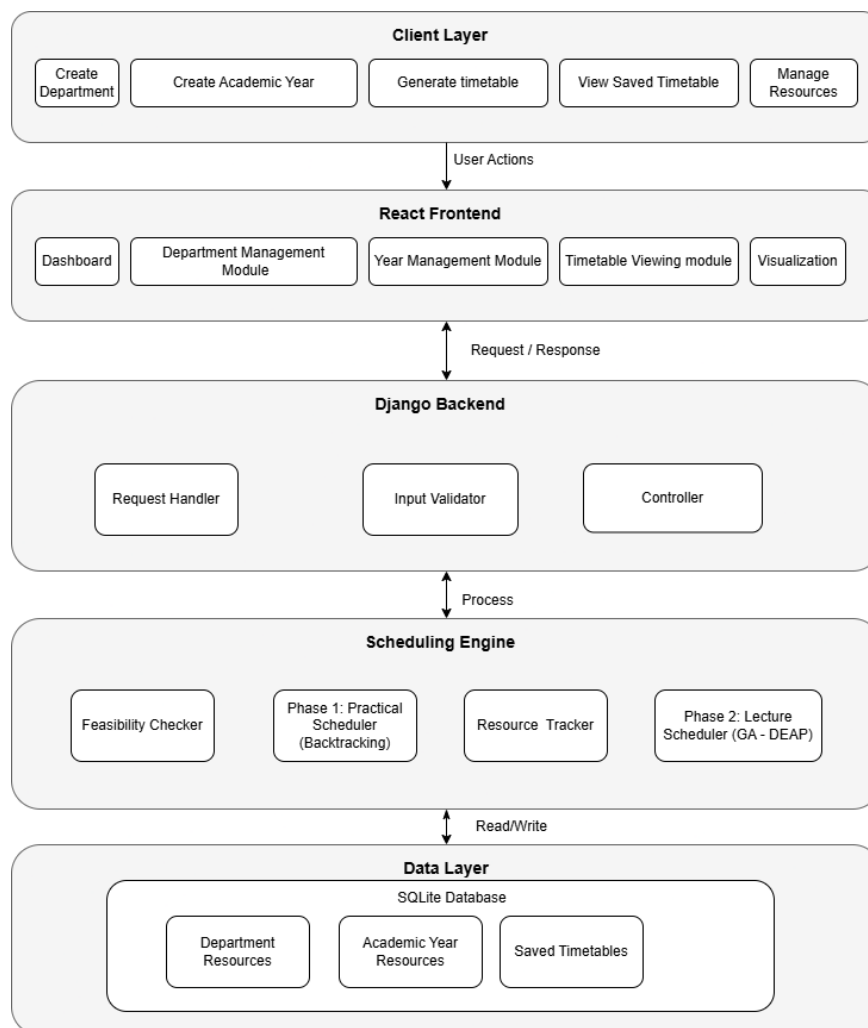


Fig. 5.1.1: Block Diagram of Proposed System Architecture

5.1.2 System Flowcharts

Flowcharts provide a step-by-step procedural view of the logic governing each phase of the timetable generator.

A) Practical Scheduling flowchart

The flowchart in Fig. 5.1.2 represents the process of practical timetable generation using a backtracking-based approach. The system begins with a feasibility check to ensure that the available resources are sufficient to generate a valid schedule. If feasible, the algorithm iterates through each academic year and assigns practical sessions one time slot at a time. For each slot, the system attempts to allocate practicals to all batches by selecting suitable combinations of subject, faculty, and laboratory. Each assignment is validated against hard constraints such as resource conflicts and capacity limits. If a valid assignment is found, it is placed and the process continues for the remaining batches. If no valid assignment is possible, the system performs backtracking by revisiting previous allocations and trying alternative options. This process repeats until all slots for a year are successfully assigned. The algorithm then proceeds to the next academic year. If all years are scheduled without conflicts, the final practical timetable is generated; otherwise, the system reports failure when no feasible solution exists.

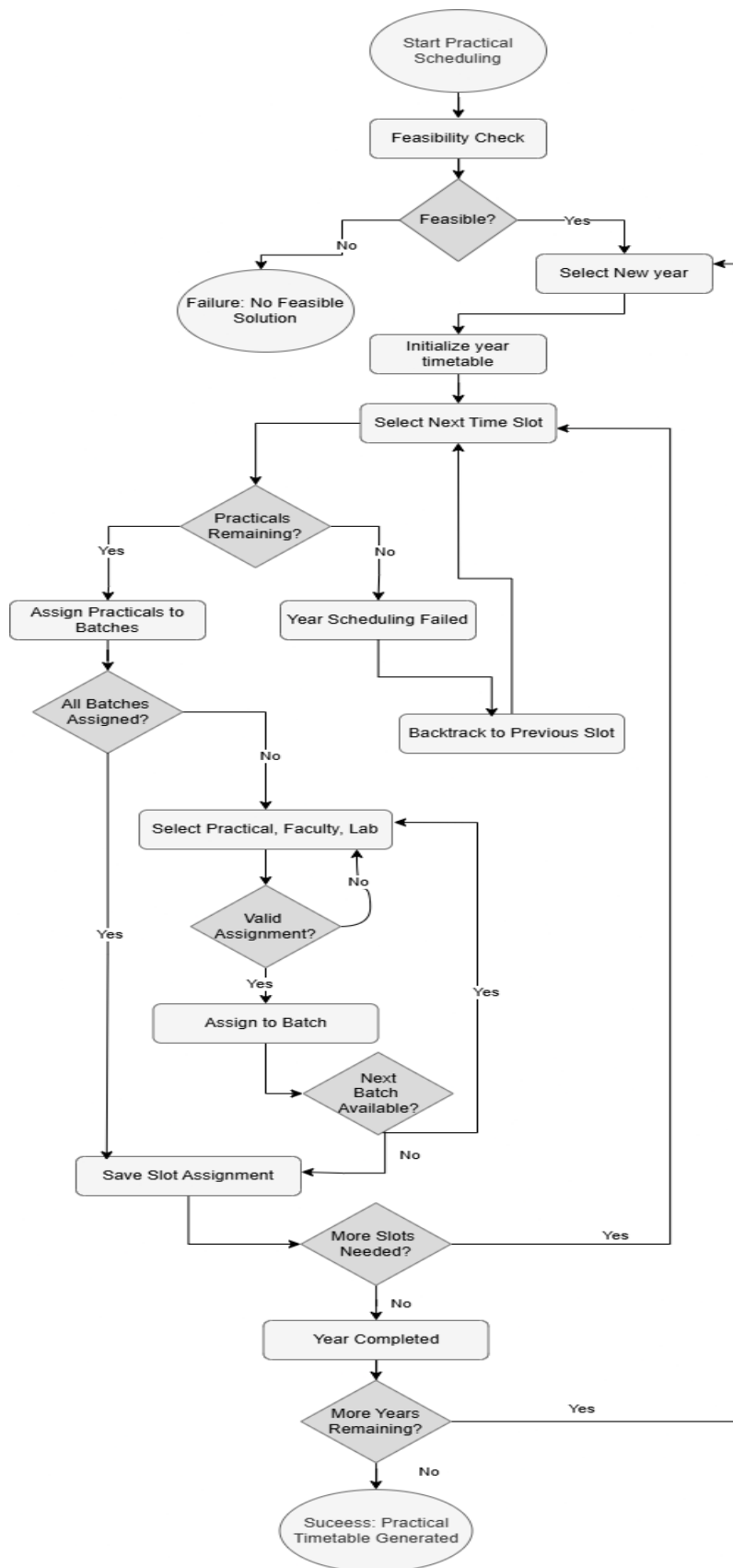


Fig. 5.1.2: Flowchart of Practical Scheduling using Backtracking

B) Lecture Scheduling Flowchart

The flowchart in Fig. 5.1.3 represents the Genetic Algorithm used for lecture scheduling in Phase 2. After practical sessions are fixed in Phase 1, the algorithm operates only on the remaining available time slots. It begins by initializing a population of possible lecture timetables, where lectures are assigned to these free slots. Each solution is evaluated using a fitness function based on constraint satisfaction and timetable quality. If an acceptable solution is found, the algorithm terminates. Otherwise, selection, crossover, and mutation operations are applied to generate improved solutions. This process repeats iteratively until an optimal or satisfactory timetable is obtained.

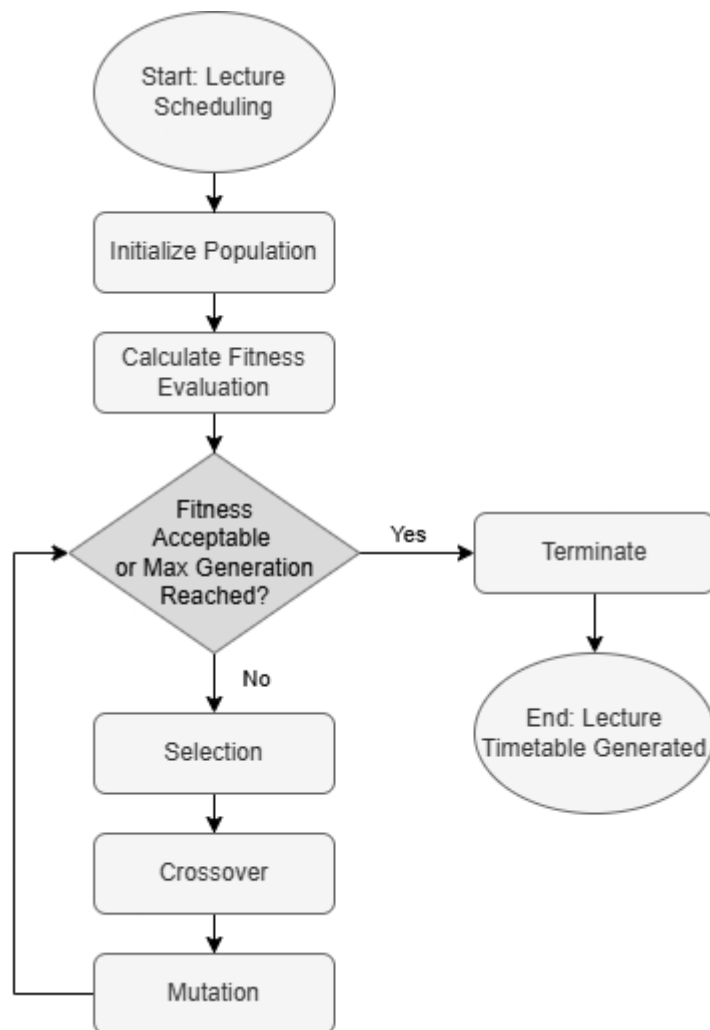


Fig. 5.1.3: Flowchart of Lecture Scheduling using GA

C) Combined flowchart

The flowchart illustrates the overall process of timetable generation using a hybrid approach. The system begins by collecting common input data such as teachers, classrooms, laboratories, and scheduling parameters. It then gathers year-wise data including subjects, student strength, and assigned faculty. After input collection, a feasibility check is performed to ensure that the available resources are sufficient to generate a valid timetable. If feasible, practical sessions are scheduled first using a backtracking algorithm to satisfy strict constraints. Once practicals are successfully assigned, lecture sessions are scheduled in the remaining time slots using a Genetic Algorithm. Finally, both practical and lecture schedules are combined to generate the complete timetable, which is then displayed to the user.

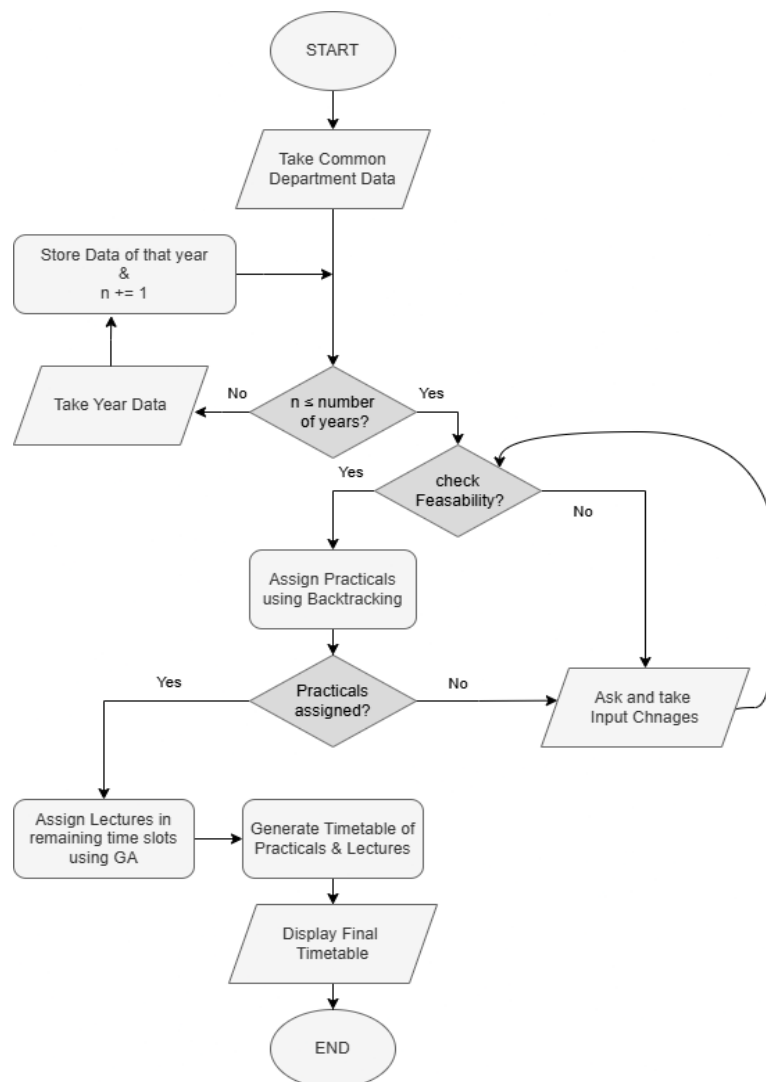


Fig. 5.1.3: Flowchart of Entire process

5.2 Methodology

5.2.1 Two-Phase Scheduling Strategy

The system adopts a two-phase scheduling approach to manage complexity effectively. In the first phase, practical sessions are scheduled using a backtracking algorithm, as they involve strict constraints such as laboratory capacity, batch synchronization, and resource availability. Once a feasible practical timetable is generated, these slots are fixed.

In the second phase, lecture sessions are scheduled in the remaining available time slots using a Genetic Algorithm. The algorithm iteratively improves solutions based on a fitness function until an acceptable timetable is obtained.

5.2.2 Justification of the Approach

A single-stage Genetic Algorithm struggles with highly constrained components like practical scheduling, leading to increased search space and slower convergence. By separating the problem, the constrained practical scheduling is handled deterministically using backtracking, while the less constrained lecture scheduling is optimized using a Genetic Algorithm. This reduces complexity and improves overall efficiency.

5.2.3 Constraint Modelling

To generate a valid timetable, the system considers two types of constraints: hard constraints and soft constraints. Hard constraints are mandatory and must always be satisfied.

Global Hard Constraints:

Applied on both lecture and practical scheduling.

- A teacher cannot be assigned to more than one session in the same time slot.
- A classroom or laboratory cannot host more than one session at the same time.
- A student cannot attend more than one session simultaneously.
- Room or laboratory capacity must be greater than or equal to the number of assigned students.
- Once a subject or practical is assigned to a teacher, the same teacher must handle it for the entire timetable.

Practical-Specific Hard Constraints:

The following constraints apply only to practical sessions.

- If a practical slot is assigned to an academic year, all batches of that year must have practical sessions.

- Once a laboratory is allocated to a batch, the same laboratory is used for all practical sessions of that batch.
- The required weekly practical hours for each subject must be satisfied exactly.
- A maximum of two practical sessions can be assigned to a batch in a single day.

Soft Constraints:

Applied only on lectures. Soft constraints are not mandatory but help improve the quality of the timetable. Violations of these constraints are penalized during optimization.

- Minimising free slots between lectures
- Maintaining balanced teaching loads for faculty members
- Distributing lectures evenly across the week

5.3 Summary

This chapter presented the system architecture, tools, scheduling methodology, and constraint modelling of the timetable generator. The hybrid approach combining backtracking and Genetic Algorithm ensures efficient and conflict-free timetable generation. The next chapter evaluates system performance and results.

CHAPTER 6

RESULTS & DISCUSSION

6.1 System Testing and Evaluation

6.1.1 Functional Testing

The system was tested to ensure that all functionalities operate correctly. The timetable generator successfully created schedules for multiple academic years based on the provided input data.

The following observations were made:

- No teacher was assigned to more than one session at the same time.
- No classroom or laboratory conflicts were observed.
- All practical sessions were scheduled according to required hours.
- Lecture sessions were successfully assigned in available time slots.
- The system handled multiple academic years simultaneously without conflicts.

These results confirm that the system satisfies all major functional requirements.

6.1.2 Performance Analysis

The performance of the system was evaluated by measuring the execution time for different problem sizes. The total execution time includes both practical scheduling (backtracking) and lecture scheduling (Genetic Algorithm).

Academic Year	Practical Events	Lecture Events	Problem Size	Practical Scheduling Time (s)	Lecture Scheduling Time (s)	Total Time (s)
Only Year 1	15	15	30	0.12	1.45	1.57
Only Year 2	9	21	30	0.25	1.68	1.93
Only Year 3	12	13	25	0.08	1.41	1.42
All Years	36	49	85	0.28	3.57	3.85

Table 6.1: Execution time Table

The results show that the system generates complete timetables within a few seconds, even as the problem size increases. Practical scheduling is relatively faster due to the deterministic nature of backtracking, while lecture scheduling takes more time due to the iterative optimization process

of the Genetic Algorithm. Overall, the system demonstrates good scalability for medium-sized datasets.

6.2 Analysis of Results

The results demonstrate that the hybrid scheduling approach is effective for solving the timetable generation problem. The backtracking algorithm efficiently handled practical scheduling, which involves strict constraints such as laboratory allocation, batch synchronization, and faculty availability. Once practical sessions were fixed, the Genetic Algorithm successfully optimized lecture scheduling in the remaining time slots, providing flexibility and improving timetable quality.

The system generated conflict-free timetables within a short execution time, even as the problem size increased, indicating good scalability. Compared to a single-stage Genetic Algorithm, the two-phase approach reduces the search space and improves convergence speed, resulting in faster and more efficient timetable generation.

However, fixing practical sessions in the first phase slightly reduces flexibility in lecture scheduling, as lectures must be adjusted around predefined slots. Despite this limitation, the system performs efficiently for medium-sized datasets and produces reliable, high-quality timetables.

6.3 Screenshots of Working Model

Homepage: SmartTimetable homepage interface displaying the hero section and scheduling highlights.

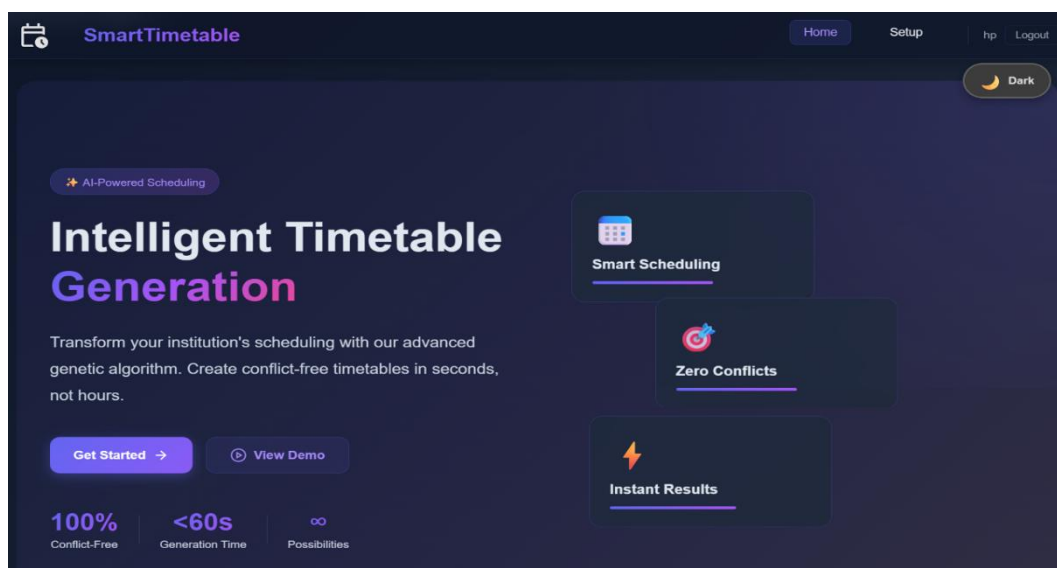


Fig. 6.3.1: Homepage

Creating Department: When user clicks a new department it leads them to this page, where users are able to enter all the departmental resources such as timeslots, teachers, classrooms, labrooms.

The screenshot shows the 'SmartTimetable' application interface for 'DEPARTMENT SETUP'. The top navigation bar includes 'Home', 'Setup', 'hp', and 'Logout' buttons, along with a 'Dark' theme toggle. The main form is divided into several sections:

- DEPARTMENT SETUP:** Contains fields for 'Department name' (IT), 'Start time' (09:15 AM), 'End time' (04:15 PM), and 'Number of days' (5).
- RECESS:** Displays two recess slots with 'Recess Start Time' and 'Recess End Time' fields, each with a 'Delete' button. A '+ New Recess' button is at the bottom.
- TEACHERS:** Includes a text area to 'Add multiple teachers (one per line)' with a list of names (Ms Dipali Tupte, Mr Abhaysinh Ghosale, Dr Rahul Patil) and an 'Add All Teachers' button. Below are individual teacher entries with 'Teacher Name' and 'Delete' buttons, and a '+ New Teacher' button.
- CLASSROOMS:** Features a text area to 'Add multiple classrooms (one per line; RoomId, Capacity)' with a list of room IDs and capacities (CR201, 120; CR202, 80) and an 'Add All Classrooms' button. Below are individual classroom entries with 'Classroom ID', 'Capacity', and 'Delete' buttons, and a '+ New Classroom' button.
- LAB ROOMS:** Includes a text area to 'Add multiple labs (one per line; LabId, Capacity)' with a list of lab IDs and capacities (LAB7, 40; LAB9, 30) and an 'Add All Labs' button. Below are individual lab room entries with 'Lab Room ID', 'Capacity', and 'Delete' buttons, and a '+ New lab' button.

A large 'Save' button is located at the bottom of the form. The footer indicates '© 2026 SmartTimetable. All rights reserved.'

Fig. 6.3.2: Department details input form

Department Dashboard: After department has been created successfully, main dashboard is shown to users that displays all the details and a header with “Manage years”, “Edit Department”, “Manage Timetables” and “Delete” button.

SmartTimetable Home Setup hp Logout Dark

Department Setup
Manage your departments, teachers, classrooms and lab rooms

+ Add Department

Department Name: IT Manage years Edit Department Manage Timetables Delete

3 TOTAL YEARS 12 TOTAL TEACHERS 3 TOTAL CLASSROOMS 6 TOTAL LABS

View Details ^

Start-time: 9 a.m.
End-time: 4:15 p.m.
Number of days in week: 5
Number of hours in one lecture: 1
Number of hours in one practical: 2
All possible lecture time slots in a day: [09:00, '10:00', '11:15', '12:15', '13:15', '15:15']
All possible practical time slots in a day: [09:00, '11:15']

RECESSES:

NO.	START TIME	END TIME
1	11 a.m.	11:15 a.m.
2	2:15 p.m.	3:15 p.m.

YEARS

NO.	YEAR NAME	TOTAL STUDENTS
1	Secondyear	55
2	Third Year	60
3	Final Year	60

ADDED TEACHERS:

NO.	TEACHER NAME
1	Ms. Maya Patil
2	Dr. Arun Saxena
3	Ms. Tanvi Patil
4	Mr. Manthan Surti
5	Mr. Abhaysinh Bhosale
6	Ms. Abhira Bainik
7	Mr. Pankaj Kumar
8	Ms. Pooja Patil
9	Ms. Dipali Tupe
10	Dr. Eknath Patil
11	Dr. Manish Rana
12	Mr. Gautam Jha

ADDED CLASS ROOMS:

NO.	CLASSROOM NUMBER	CAPACITY
1	A-12	65
2	A-302	70
3	A-24	60

ADDED LABS:

NO.	LAB NUMBER	CAPACITY
1	Lab1	25
2	Lab2	25
3	Lab3	25
4	Lab4	25
5	LabA-110	30
6	LabA-111	30

Fig. 6.3.3: Department Dashboard

Manage years page— shows users all the details of all the years added inside that department sequentially.

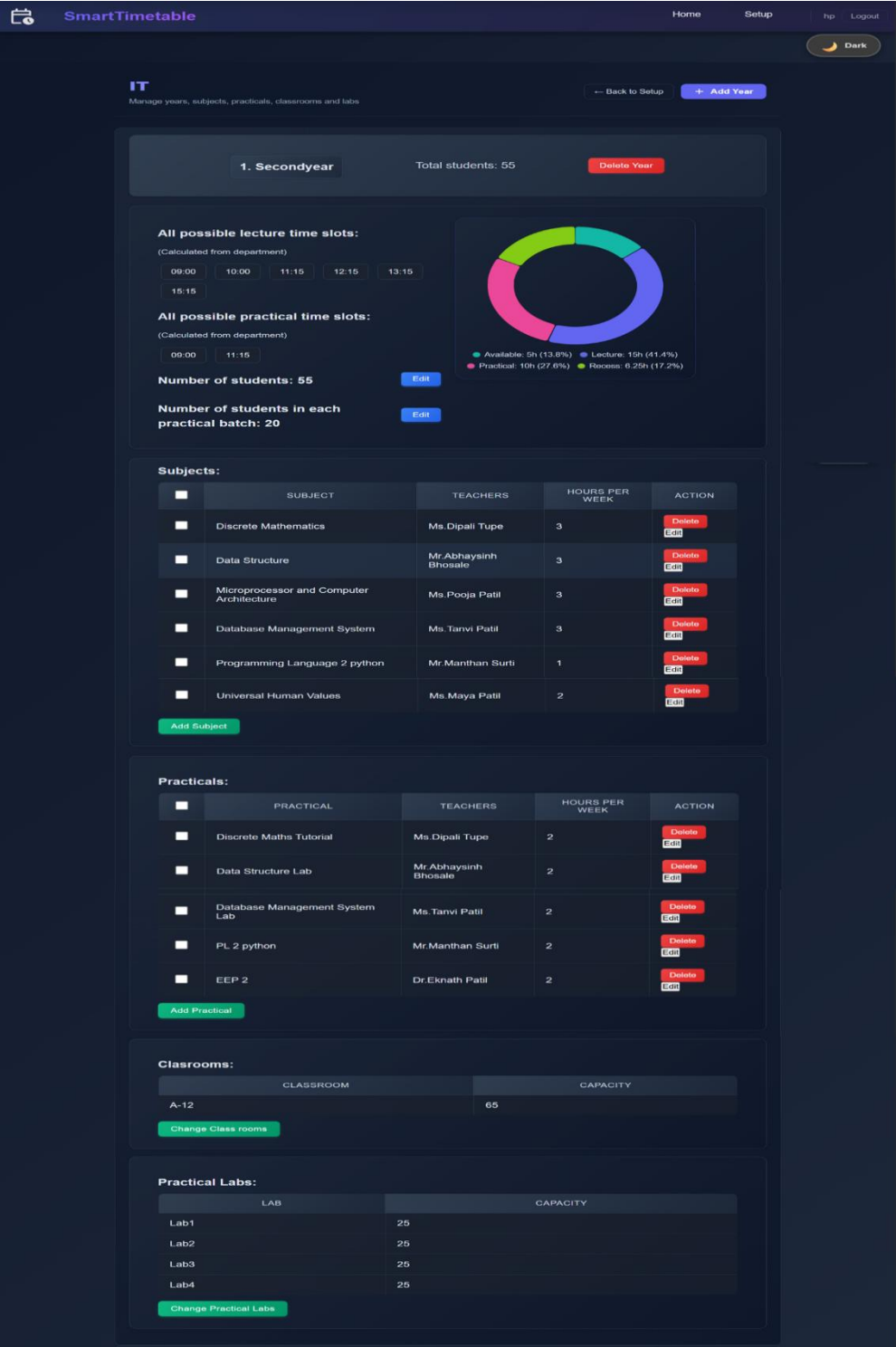


Fig. 6.3.4: Manage years page

Manage Timetable page – displays the saved timetables and a generate new timetable button

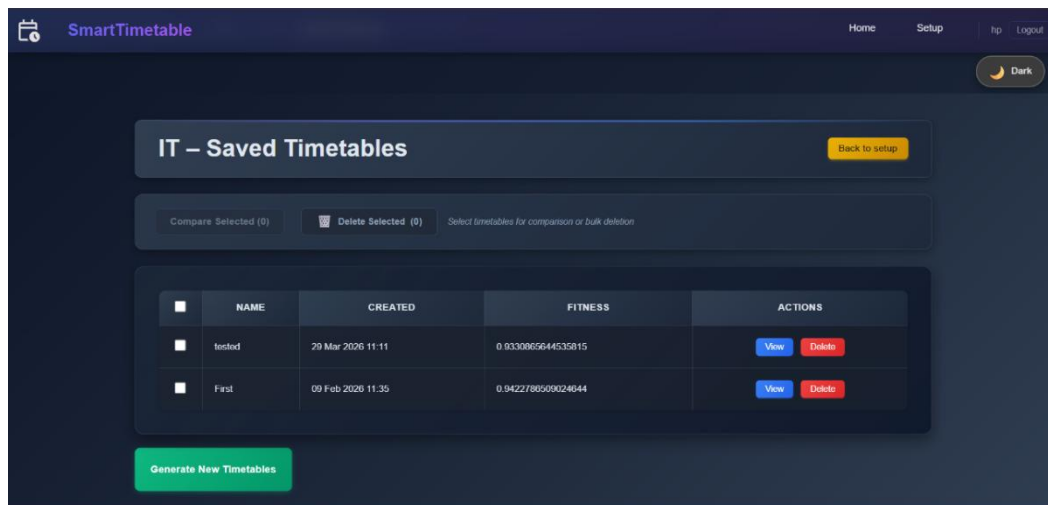


Fig. 6.3.5: Manage Timetable Page

Generated Timetable Page – Shows the generated timetable along with faculty workload across a week.



Fig. 6.3.6: Generated Timetable Page

6.4 Summary

This chapter presented the evaluation of the Timetable Generator system through functional testing, performance analysis, and output verification. The system successfully generated conflict-free timetables while satisfying all defined constraints. The execution time remained low even for larger problem sizes, demonstrating good efficiency and scalability.

The analysis confirmed that the hybrid approach using backtracking for practical scheduling and Genetic Algorithm for lecture scheduling is effective in reducing complexity and improving performance. The screenshots of the working system illustrate a complete and functional application with a user-friendly interface for managing inputs and viewing generated timetables.

CHAPTER 7

Conclusion and Future Scope

7.1 Summary of Findings

The project successfully developed an automated timetable generation system using a hybrid approach combining backtracking and Genetic Algorithm. The system efficiently generated conflict-free timetables for multiple academic years while satisfying all defined constraints.

The backtracking algorithm proved effective in handling highly constrained practical scheduling, ensuring proper allocation of laboratories, batches, and faculty. The Genetic Algorithm efficiently scheduled lectures in the remaining time slots, optimizing overall timetable quality. The results demonstrate that separating the problem into two phases reduces complexity and improves performance compared to traditional single-stage approaches.

Overall, the system achieved reliable timetable generation with minimal execution time, making it suitable for academic institutions.

7.2 Future Scope and Possible Enhancements

- 1. Multi-Department Support:** Extend the system to handle multiple departments simultaneously, enabling centralized timetable generation across the institution.
- 2. Rescheduling Functionality:** Implement dynamic rescheduling to adjust lectures or practical sessions without regenerating the entire timetable.
- 3. Deployment and Accessibility:** Deploy the system on cloud platforms for improved accessibility and scalability.
- 4. Leave Management Integration:** Incorporate faculty leave management and the ability to automatically adjust schedules.
- 5. Advanced Constraint Handling:** Support soft constraints specific to teachers, subjects, classrooms, and laboratories for more flexible scheduling.
- 6. Comparative Analysis with Standard Genetic Algorithm:** Perform a detailed comparison between the proposed two-phase hybrid approach and a traditional single-stage Genetic Algorithm where both lectures and practicals are scheduled together. The comparison can be based on execution time, constraint satisfaction, convergence speed, and timetable quality to further validate the effectiveness of the proposed method.

References

- [1] Dipesh Mittal, Hiral Doshi, Mohammed Sunasra, Renuka Nagpure (2015) "Automatic Timetable Generation using Genetic Algorithm"
- [2] Akshay puttaswamy, H M Arshad Ali Khan, Chandan S.V, Parkavi.A (2017) " STUDY ON AUTOMATIC TIMETABLE GENERATOR "
- [3] Dexter Romagueraa, Jenie Plender-Nabasa, Junrie Matiasa*, Lea Austerob (2023) " Development of a Web-based Course Timetabling System based on an Enhanced Genetic Algorithm"
- [4] Sahil Sarnaik, Tanmay More, Sanket Shinde, Dipesh Shah (2023) "Timetable Generator using Genetic Algorithm"
- [5] Kunjan KatoreRasika Lahane,Punam Motewar,Chaitali Nakhate (2024) " DYNAMIC TIMETABLE GENERATOR USING GENETIC ALGORITHM."
- [6] Ahmed, *Umar, Abdullahi ,Maliki, Anda and Kamaldeen J " Development of a Model for Generation of Examination Timetable Using Genetic Algorithm."
- [7] Mohammad A. Al-Jarrah, Ahmad A. Al-Sawalqah and Sami F. Al-Hamdan (2017) "DEVELOPING A COURSE TIMETABLE SYSTEM FOR ACADEMIC DEPARTMENTS USING GENETIC ALGORITHM "
- [8] Maram Assi, and Bahia Halawi, and Ramzi A. Haraty*(2018) " Genetic Algorithm Analysis using the Graph Coloring Method for Solving the University Timetable Problem "
- [9] Azilawati Rozaimiee, Adibah Nabihah Shafee, Nurul Anissa Abdul Hadi and Mohamad Afendee Mohamed Faculty of Informatics and Computing, Universiti Sultan Zainal Abidin, Malaysia(2017) "A Framework for University's Final Exam Timetable Allocation Using Genetic Algorithm "
- [10] Electronic Soyemi, J., Akinode, J., & Oloruntoba, S. (2017) " Electronic Lecture Time-Table Scheduler Using Genetic Algorithm."

Appendix

Technologies Used

1. **React-Based Frontend:** The frontend is developed using React, enabling dynamic and responsive user interaction. It provides structured modules for managing resources and viewing generated timetables.
2. **Django Backend Framework:** Django is used to handle request processing, input validation, and coordination between frontend and scheduling logic. It ensures secure and structured data handling.
3. **Hybrid Scheduling Approach (Backtracking + Genetic Algorithm):** A two-phase scheduling strategy is used:
 - a. Backtracking algorithm handles practical scheduling due to strict constraints
 - b. Genetic Algorithm optimizes lecture scheduling in remaining time slots
4. **Genetic Algorithm using DEAP Library:** The DEAP library is used to implement GA operations such as population generation, selection, crossover, mutation, and fitness evaluation to produce optimized lecture timetables.
5. **Occupancy Matrix using NumPy:** A NumPy-based matrix is used to track resource usage (teachers, labs, classrooms, time slots). This ensures fast conflict detection and prevents overlapping allocations.
6. **Data Visualization using Chart.js:** Visualization improves user understanding through:
 - a. 1.Pie Chart: Displays distribution of lecture, practical, recess, and free hours for each academic year.
 - b. 2.Stacked Bar Chart: Shows teacher workload per week and how it is distributed across different academic years.

Publication

The research work titled **“A Two-Phase Genetic Algorithm Approach for Academic Timetabling”** has been successfully prepared and submitted to the **I2ITCON Pune Conference 2026** under the track Artificial Intelligence and Applications. The paper proposes a hybrid scheduling approach that combines backtracking for practical session allocation with a Genetic Algorithm for lecture scheduling to ensure a conflict-free timetable across multiple academic years. This submission reflects the application of optimization techniques to solve real-world academic scheduling challenges and demonstrates the practical relevance of the proposed methodology.

Acknowledgements

We would like to express our sincere gratitude to our Principal, **Dr. Kamal Shah**, Principal of St. John College of Engineering and Management, for providing us with the facilities, support, and encouragement to complete our major project successfully. We also extend our heartfelt thanks to our Head of Department, **Dr. Arun Saxena**, HOD, Department of Information Technology, for his valuable guidance and continuous motivation throughout the project. We are deeply grateful to our Guide, **Mr. Pankaj Kumar** Assistant Professor, Department of Information Technology, for his constant support, valuable feedback, and insightful suggestions that helped us accomplish this work effectively. We also wish to thank our colleagues and friends for their cooperation and teamwork during the development of this project. Finally, we owe our deepest gratitude to our parents for their constant encouragement, moral support, and blessings, which have been our inspiration throughout this academic journey.

Raviprakash Gupta [EU2214008]

Suryanarayan Pillai [EU1224019]

Ayush Raut [EU1224004]